

Dataset E – Learning & Course Management System

Assessment Objective

This assessment evaluates your ability to build a complete MERN Full Stack application involving:

- Authentication & Authorization
- External API Integration
- Data Validation & Sanitization
- MongoDB Persistence
- Relationship Modeling
- REST API Development
- Querying & Aggregation
- Frontend Integration
- Protected Routes
- State Management
- Deployment Readiness

The assessment focuses on engineering workflows rather than isolated CRUD operations.

Problem Statement

You are required to develop a:

Learning & Course Management System

The system manages:

- Students
- Instructors
- Courses
- Enrollments
- Assessments

Students must:

1. Fetch learning dataset from external API
2. Validate and sanitize records
3. Persist valid records into MongoDB Atlas
4. Build secure REST APIs
5. Implement role-based authorization
6. Build React frontend consuming backend APIs
7. Implement analytics & dashboard features

Standard Response Structure

Success Response

```
{
  "success": true,
  "message": "Operation successful",
  "data": []
}
```

Error Response

```
{
  "success": false,
  "message": "Error message"
}
```

Section A — Authentication & Authorization

Objective

Implement JWT-based authentication and role-based access control.

Q1 — Register API

POST /auth/register

Requirements

- Register new user
- Password hashing mandatory
- Duplicate email prevention
- Store role

Roles

- admin
- instructor
- student
- coordinator

Q2 — Login API

POST /auth/login

Requirements

- JWT token generation
- Invalid credential handling
- Return authenticated user details

Q3 — Current User API

GET /auth/me

Requirements

- Protected route

- Return logged-in user details

```
"rejected": 18
}
```

Authorization Rules

Only:

- admin
- coordinator

can:

- create courses
- assign instructors
- access analytics APIs
- manage enrollments

Instructors can:

- manage assessments
- update student marks
- view enrolled students

Students can:

- enroll in courses
- view assessments
- view marks

Section B — Dataset Synchronization & MongoDB Persistence

Objective

Fetch dataset from external API, validate records, sanitize values, and persist valid records into MongoDB Atlas.

Q4 — Sync API

POST /sync

Responsibilities

- Fetch dataset from private API
- Validate records
- Normalize values
- Reject invalid entries
- Prevent duplicate insertion
- Persist valid records into MongoDB
- Return sync summary

Example Response

```
{
  "success": true,
  "totalFetched": 150,
  "inserted": 120,
  "duplicates": 12,
}
```

Required Sanitization Examples

Raw Value	Expected Value
" FULLSTACK "	"FullStack"
"ACTIVE"	"active"
" React Basics "	"React Basics"

Validation Requirements

Reject:

- invalid course references
- invalid grades
- duplicate enrollment IDs
- invalid student references
- negative marks
- invalid assessment types

Q5 — Health API

GET /health

Example

```
{
  "success": true,
  "database": "connected",
  "documentCount": 120
}
```

Section C — MongoDB Persistence, Relationships & Runtime Validation

Objective

Implement MongoDB persistence with proper relationships, validation logic, and runtime data consistency.

Required Collections & Relationships

User

Fields

- userId
- name
- email
- role
- department

- status

Course

Relationships

- instructor → User

Fields

- courseId
- title
- category
- duration
- level
- status

Enrollment

Relationships

- student → User
- course → Course

Fields

- enrollmentId
- enrolledAt
- progress
- completionStatus

Assessment

Relationships

- course → Course
- student → User

Fields

- assessmentId
- title
- type
- marks
- grade
- submittedAt

Activity Log

Relationships

- user → User

Fields

- action
- module
- timestamp

Section D — CRUD & Workflow APIs

Objective

Implement REST APIs with proper validation and workflow handling.

Q6 — User APIs

GET /users
GET /users/:id

Q7 — Course APIs

POST /courses
GET /courses
GET /courses/:id
PATCH /courses/:id
DELETE /courses/:id

Q8 — Enrollment APIs

POST /enrollments
GET /enrollments
GET /enrollments/:id
PATCH /enrollments/:id
DELETE /enrollments/:id

Q9 — Assessment APIs

POST /assessments
GET /assessments
GET /assessments/:id
PATCH /assessments/:id
DELETE /assessments/:id

Workflow Rules

- Students cannot enroll twice in same course
- Inactive courses cannot accept enrollments
- Assessments cannot exceed maximum marks
- Only assigned instructors can update marks
- Completed courses cannot accept new assessments

Section E — Learning Workflow APIs

Objective

Implement protected workflow APIs.

Q10 — Assign Instructor

PATCH /courses/:id/assign

Requirements

- Only admin/coordinator allowed
- Instructor must exist
- Archived courses cannot accept assignments

Q11 — Update Course Status

PATCH /courses/:id/status

Allowed Status

- active
- ongoing
- completed
- archived

Workflow Rules

- Archived courses cannot move back to active
- Completed courses cannot accept enrollments
- Students cannot modify course status
- Instructors cannot archive courses directly

Section F — Filtering, Search & Pagination

Objective

Implement scalable querying APIs.

Q12 — Course Filters

Examples

GET /courses?category=Web

GET /courses?level=advanced

GET /courses?status=active

Q13 — Enrollment Filters

Examples

GET /enrollments?completionStatus=completed

GET /enrollments?progressMin=70

Q14 — Pagination & Search

Examples

GET /courses?page=1&limit=10

GET /courses?search=react

Combined Query Example

GET /courses?

category=Web&

status=active&

page=2&

limit=5

Section G — Analytics & Aggregation APIs

Objective

Implement MongoDB aggregation APIs.

Q15 — Learning Analytics

GET /analytics/learning

Return

- total students
- active courses
- completed enrollments
- average completion percentage

Q16 — Course Analytics

GET /analytics/courses

Return

- course-wise enrollment count
- highest completion rate
- active instructor count
- archived courses count

Q17 — Assessment Analytics

GET /analytics/assessments

Return

- average marks
- grade distribution
- top performing students
- failed assessments count

Section H — Frontend Integration

Objective

Build React frontend consuming backend APIs.

Mandatory Frontend Features

Authentication

- Login page
- JWT token handling
- Protected routes
- Role-based rendering

Dashboard

Display:

- learning analytics cards
- recent enrollments
- course summaries
- assessment statistics

Student Module

- enroll courses
- view assessments
- track progress
- view grades

Instructor Module

- manage assessments
- update marks
- view enrolled students
- manage courses

Mandatory State Management

Use:

- Context API
- Reducer

Section I — Frontend Evaluator Compliance, Global State & Protected Routes (5 Marks)

Objective

Evaluate frontend architecture.

Requirements

Implement:

- Protected routes
- Role-based navigation
- Reducer logic
- Global state management
- Required data-testid attributes
- Dynamic UI updates based on global state

Mandatory Global State

Your application must maintain global state for:

- authenticated user
- JWT token
- users
- courses
- enrollments
- assessments
- filters/search values
- analytics/dashboard data

Use:

- Context API
- Reducer

Mandatory data-testid Attributes

Authentication

Component	data-testid
Login form	login-form
Email input	email-input
Password input	password-input
Login button	login-btn
Logout button	logout-btn

Navigation

Component	data-testid
Navbar	navbar
Dashboard link	dashboard-link
Courses link	courses-link
Enrollments link	enrollments-link
Assessments link	assessments-link
Students link	students-link

Dashboard

Component	data-testid
Analytics container	analytics-container
Total students card	total-students-card
Active courses card	active-courses-card

Completion rate card	completion-rate-card
Assessment chart	assessment-chart
Recent enrollments section	recent-enrollments

Course Module

Component	data-testid
Course table	course-table
Course search input	course-search
Course filter dropdown	course-filter
Course row	course-row

Enrollment Module

Component	data-testid
Enrollment table	enrollment-table
Enroll button	enroll-btn
Enrollment row	enrollment-row
Pagination next button	pagination-next
Pagination previous button	pagination-prev

Assessment Module

Component	data-testid
Assessment table	assessment-table
Add assessment button	add-assessment-btn
Assessment row	assessment-row

Section J — Frontend Evaluator Compliance & Deployment (10 Marks)

Objective

Validate deployment readiness, frontend evaluator compatibility, and runtime UI behavior using automated Playwright-based evaluation.

Mandatory Frontend Behavior

Your frontend must:

- consume your own backend APIs
- update UI dynamically after CRUD operations
- maintain JWT token in global state
- maintain authenticated session after refresh
- restrict unauthorized route access
- render UI based on user role

Mandatory React Router Routes

Students must implement the following routes using React Router DOM.

Route	Purpose
/login	Authentication
/dashboard	Analytics dashboard
/courses	Course management
/enrollments	Enrollment management
/assessments	Assessment management
/students	Student management
/profile	Current user profile

Global State Exposure Requirement

To support automated Playwright evaluation, students must expose global application state using:

window.appState

The global state object must dynamically reflect the current application state.

Mandatory window.appState Structure

```
window.appState = {
  authUser,
  token,
  users,
  courses,
  enrollments,
  assessments,
  filters,
  analytics
}
```